

The Alberta Buck - Notes (DRAFT v0.1)

A Unified Commitment-Pool Architecture for Wormhole, Cash, and Cheques

Perry Kundert

2026-04-18



The Alberta Buck *Wormhole* (Wormhole) and *Cash* (Cash) designs both need a privacy-preserving bridge from an on-chain balance to a later on-chain mint. In the current series both designs supply their own mechanics: the Wormhole uses per-certificate burn addresses and `_mint_noincrease`; Cash uses a Zerocash-style commitment tree with Mode A/B/C spend predicates (bearer, stealth-targeted, programmable). The overlap is nearly complete – both reduce to "burn into a pool, mint out by SNARK" – and maintaining two architectures doubles the verifier surface, the circuit count, and the audit scope without buying additional functionality.

This document collapses the two into one primitive. A single Merkle tree of note commitments and a single nullifier set replace both the burn-address family and the Cash note pool. Three *Identity bindings* parameterise the commitment:

- **Type 1 – Encrypted Issuer Identity.** Spend predicate requires the depositor's registered ElGamal credential to encrypt the same identity point the issuer's credential encrypts. Subsumes the v1 Wormhole: the issuer and the eventual depositor must share an Identity scalar, but may use unlinkable key pairs.
- **Type 2 – Public Issuer Identity.** Spend predicate requires the depositor to be any registered identity; the issuer's public identity is bound to the note in plaintext. A cheque whose maker is known, whose bearer is not.
- **Type 3 – Stealth-Targeted to Recipient.** Spend predicate requires the depositor to hold the secret of a stealth public key derived from the recipient's registered identity. A cheque made out to a specific recipient, retrievable only by them.

Demurrage needs no new machinery. The Note Pool *is* simply the BUCK ERC-20 contract's own balance – a regular BUCK account accumulating BUCK-age like any other account. The

contract tracks just one additional scalar beyond the Merkle tree and nullifier set: F , the sum of face values of live (unspent) notes. When a note of face v is redeemed, the contract delivers $v_{\text{delivered}} = v \cdot (B - \kappa A)/F$ BUCK to the recipient, where B and A are the contract account's balance and accumulated BUCK-age in the existing ledger – quantities already maintained by the ERC-20 demurrage machinery for every account. Donations sent to the contract address (deliberate, philanthropic, or accidental) raise B without raising F , so subsequent redemptions deliver more than the equilibrium pool-rate – possibly more than face value. Conservation is automatic: demurrage is never actually transferred; the Jubilee balance grows autonomously per block at rate $\kappa \cdot \text{totalSupply}$, and every account (pool included) reduces its spendable balance by its own κA on outgoing transfers. No Little's Law, no continuous Jubilee stream, no separate pool obligation.

The consequence most relevant to users: only notes where the issuer cannot later race the holder to spend are safe for long-term holding. This is **Type 1** (issuer = depositor, same Identity, no third party knows the note secret) and **Type 3** (the issuer's knowledge of the note secret does not suffice to spend – only the stealth-derived secret can). Type 2's public-issuer attribution comes at the cost of retained issuer-spend capability, restricting it to short-lived use cases (cheques presented within hours or days, not stored for years). This is an intrinsic cryptographic property, not a policy choice. (PDF, Text)

DRAFT

Contents

1	The Redundancy Between Wormhole and Cash	5
1.1	The Problem Both Solve	5
1.2	The Two Current Mechanisms	5
1.3	The Overlap	5
1.4	Maintenance Cost of Duality	6
2	The Single Primitive: Notes in One Global Pool	6
2.1	The Note Contract State	6
2.2	The Note Commitment	6
2.3	The Note Nullifier	7
2.4	Lifecycle	7
3	Three Identity Bindings	9
3.1	Type 1: Encrypted Issuer Identity ("Wormhole")	9
3.2	Type 2: Public Issuer Identity ("Signed Cheque")	9
3.3	Type 3: Stealth-Targeted to Recipient ("Addressed Cheque")	10
3.4	Comparison of the Three Bindings	10
4	The Interaction Matrix: Note Binding x Account Mode	11
5	Demurrage at the Pool Level	13
5.1	Reminder: How BUCK Demurrage Actually Works	13
5.2	The Pool Account's Demurrage	13
5.3	The Deposit Formula	14
5.4	Donations and Negative Effective Demurrage	14
5.5	Conservation is Automatic	14
5.6	Numeric Example	15
5.7	Who Bears the Demurrage, Really	15
6	Long-Term Holding Safety: The Printer-Trust Distinction	16
6.1	The Printer-Trust Model	16
6.2	Type 1 (Encrypted Issuer Identity): Safe	16
6.3	Type 2 (Public Issuer Identity): Unsafe for Long-Term Storage	16
6.4	Type 3 (Stealth-Targeted to Recipient): Safe	16
6.5	Summary: Which Notes Can Be Saved?	17
7	Required Identity Cryptography	18
7.1	Components Already Present	18
7.2	New Primitive: The Note Commitment Hash	18
7.3	New Primitive: The Nullifier PRF	18
7.4	Re-Used Primitive: Theorem 7 (Stealth Address)	19
7.5	The Re-Encryption Operation at Mint	19
7.6	The Re-Encryption Operation at Spend (Type 1 Only)	19
7.7	The Schnorr PoK Operation at Spend (Type 2 and Type 3)	19
7.8	Summary: New Cryptographic Components	19

8	The SNARK Circuits	20
8.1	Mint Circuit	20
8.2	Spend Circuit (Type 1 – Encrypted Issuer)	20
8.3	Spend Circuit (Type 2 – Public Issuer)	21
8.4	Spend Circuit (Type 3 – Stealth-Targeted)	21
8.5	Circuit Engineering Budget	22
8.6	Common Circuit Gadgets	22
9	Migration from v1 Wormhole and BUCK Cash	23
9.1	Wormhole v1 to Notes	23
9.2	BUCK Cash to Notes	23
9.3	v1 Coexistence During Transition	23
10	What Gets Dropped: Pure Bearer	24
10.1	Why Mode A Is Absent	24
10.2	The Physical-Token Escape Hatch	24
10.3	Use-Case Coverage Without Mode A	24
11	Security Assumptions and Theorems Used	25
11.1	Demurrage Conservation as a Contract Invariant	25
12	Implementation Scope	26
12.1	Smart Contract Additions	26
12.2	SNARK Circuits	26
12.3	Wallet & UX	26
12.4	Cross-Checks and Audit	27
13	Open Questions	28
13.1	Donation-Driven Premia and Their Incidence	28
13.2	Stranded Value on Lost Notes	28
13.3	Integer-Arithmetic Rounding	28
13.4	Predicate Programmability vs. Circuit Simplicity	28
13.5	Universal Spend Circuit vs. Three	28
13.6	Regulatory Interaction with Redemption Premia	29
14	Summary	30

1 The Redundancy Between Wormhole and Cash

The v1 Wormhole (Wormhole) and the BUCK Cash proposal (Cash) both solve the same core problem by different mechanical routes.

1.1 The Problem Both Solve

Each requires a cryptographic pipeline that:

1. Accepts a burn of an existing on-chain balance
2. Creates a durable, privacy-preserving token (certificate or note) referencing the burn
3. Permits a later on-chain mint that can be proved (in zero knowledge) to correspond to some prior burn
4. Prevents double-spending via a global nullifier
5. Preserves BUCK's demurrage economics (the minted amount must reflect accumulated demurrage across the burn-to-mint interval)
6. Composes with the existing Identity system (so that the mint lands in a properly registered, KYC-verified account)

1.2 The Two Current Mechanisms

Wormhole v1 (burn-address model). Each certificate is anchored by a distinct EVM address that receives a fixed-denomination transfer. The burn is an ordinary `transfer` call; the ZK proof binds the destination mint to the source burn through the `_mint_noincrease` primitive that transfers balance and BUCK-age into the destination without adjusting `totalSupply`. Each burn address costs an identity registration, an `approveFor` handshake, and a state-trie slot.

BUCK Cash (note-pool model). A contract-managed Merkle tree of note commitments replaces the per-burn-address state anchor. A single burn transaction appends N commitments to the tree and emits the summed value into the contract reserve. Each spend is a SNARK proving inclusion of some commitment and emission of a unique nullifier. The spend *predicate* inside the commitment distinguishes bearer (Mode A), stealth-targeted (Mode B), and programmable (Mode C) notes.

1.3 The Overlap

Functionally these are nearly identical. Both provide commit-nullify privacy; both bind the burn to a later mint through a ZK relation; both produce one nullifier per unit redeemed; both depend on the Identity system for the minting-account side. The divergence is cosmetic:

Wormhole v1 anchor	Cash note-pool anchor
<code>addr_burn</code> (state trie leaf)	<code>cm_i</code> (Merkle tree leaf)
<code>_mint_noincrease</code> to destination	<code>mint</code> directly (supply unchanged by note pool)
Per-burn <code>approveFor</code> + <code>approve</code>	No per-note approval (spend predicate)
Fixed-denomination cohort anonymity set	Global pool anonymity set
Spend relation: CP re-encryption (Theorems 3-6)	Spend relation: predicate-specific (Theorems 4, 7)

What neither design fully expresses is that the *spend relation* is orthogonal to the pool structure: Chaum-Pedersen re-encryption, stealth-address Schnorr, and arbitrary predicates can *all* coexist

as different spend circuits on the same note pool. If we take this seriously, burn addresses and `_mint_noincrease` become redundant: the note pool absorbs the Wormhole as one spend-predicate family among many.

1.4 Maintenance Cost of Duality

Keeping both mechanisms in production doubles:

- **Verifier code.** Two Merkle-style lookups, two nullifier-set discipline rules, two edge-case sets for the auditor.
- **SNARK circuits.** The Wormhole CP-re-encryption circuit and the Cash bearer/targeted/programmable circuits do not share skeleton code; the proof systems compile separately.
- **Demurrage bookkeeping.** The Wormhole’s per-certificate BUCK-age is computed against EVM storage; the Cash note’s per-commitment $(t_{\text{print}}, \text{age}_0)$ is computed in the SNARK. Two distinct conservation invariants.
- **Identity integration.** Wormhole requires `approveFor` against the burn-address pseudo-account; Cash predicates compose differently with Schnorr authorisations.
- **Documentation.** Two designs to teach, two attack tables to maintain, two sets of open questions.

A unified architecture reduces all of this to one audit, one circuit family, one invariant.

2 The Single Primitive: Notes in One Global Pool

The unified design replaces both mechanisms with a single primitive.

2.1 The Note Contract State

The BUCK contract gains exactly three new pieces of state:

State	Type	Purpose
<code>noteTree</code>	Merkle tree of commitments	Cryptographic anchor for every outstanding note
<code>nullifiers</code>	Set of used nullifiers	Double-spend prevention
<code>noteFaceSum</code>	F in $\mathbb{Z}$	Sum of face values of all outstanding notes

That is all. The pool’s nominal BUCK balance B and accumulated BUCK-age A are not tracked as separate state: *the pool is just a BUCK account*, specifically the ERC-20 contract’s own address, and B, A are exactly the balance and BUCK-age the ERC-20 ledger already maintains for it. No lazy integration, no shadow aggregates, no separate `poolT` timestamp. BUCK transferred to the contract address (via mint, donation, or accident) joins B through the normal ERC-20 path; BUCK transferred out (via redemption) leaves through the normal ERC-20 path with the normal demurrage accounting.

2.2 The Note Commitment

A note commitment is a Poseidon hash of the fields the SNARK will later open:

$$cm = H_{\text{cm}}(\text{type}, v, \rho, \text{id}_{\text{payload}}, \text{predicate})$$

with:

- **type** $\in \{1, 2, 3\}$: the Identity-binding mode
- **v** $\in \mathbb{Z}_q$: face value
- **ρ** : per-note randomness (source of the nullifier)
- **id_{payload}**: Identity-binding data (see next section)
- **predicate**: optional additional spend predicate (for future Mode C programmability – not elaborated in this document; a single bit **predicate** = None= keeps the SNARK to the three base types)

No per-note timestamp or baked-in BUCK-age: demurrage is computed at redemption time from the *pool's* account state (B, A) , not from the individual note's life history, so no per-note age fields need to ride in the commitment.

The commitment is inserted as a leaf of **noteTree** at mint time.

2.3 The Note Nullifier

The nullifier is a pseudo-random function of the note's secret witness material:

$$nf = \text{PRF}_{\text{key}}(\rho)$$

where **key** is the spend-side secret specific to the note type (the issuer's *sk* in Type 1, any registered spender's *sk* in Type 2, the stealth *sk** in Type 3). The PRF is instantiated via Poseidon in the SNARK-friendly setting.

The nullifier is emitted on deposit; double-spend is rejected by set-membership check.

2.4 Lifecycle

mint (burn):

```
# Standard ERC-20 transfer from caller to contract address.
# Caller pays their own outgoing-transfer demurrage through the
# normal BUCK accounting path. Contract balance B gains v; its
# accumulated BUCK-age A starts earning on the new v from here.
transfer v BUCK from caller to contract
for each of {cm_1, ..., cm_N}:
    append cm_i to noteTree
noteFaceSum += sum(v_i)
```

deposit (mint-out):

```
verify SNARK:
    - Merkle inclusion of some cm in noteTree
    - Knowledge of (v, rho, id_payload, predicate)
    - Predicate satisfied by witness (type-specific)
    - nullifier correctness
    - v is a public output of the SNARK
require nf not in nullifiers
nullifiers += nf
# Contract reads its own balance and BUCK-age from the ERC-20 ledger:
B = balanceOf(contract)
```

```

A = buckAgeOf(contract)
delivered = v * (B - kappa * A) / noteFaceSum
# Standard ERC-20 transfer from contract to recipient.  Contract's
# demurrage accounting on (B, A) is updated by the normal BUCK path;
# no special-case bookkeeping.
transfer delivered BUCK from contract to recipient
noteFaceSum -= v

```

The deposit amount is *less than face* by the pool's current demurrage-per-face-unit $\kappa A/F$, but may equal or exceed v if donations have pushed $B > F$ (see "Donations and Negative Effective Demurrage" below). No Jubilee payment is emitted from inside the deposit: the Jubilee grows autonomously on every block at rate $\kappa \cdot \text{totalSupply}$, and the contract's share of that growth is already reflected in A .

DRAFT

3 Three Identity Bindings

Every note commits to one of three *Identity-binding* payloads. The binding determines who can spend the note and what observers learn about the issuer.

3.1 Type 1: Encrypted Issuer Identity ("Wormhole")

Identity payload.

$$\text{id}_{\text{payload}}^{(1)} = E_{\text{issuer}} = (R_i, C_i) = (r_i G, M_{\text{issuer}} + r_i \cdot pk_{\text{issuer}})$$

where pk_{issuer} is the issuer's own identity public key at mint time, $M_{\text{issuer}} = m_{\text{issuer}} \cdot G$, and r_i is fresh randomness. This is a standard ElGamal encryption of the issuer's Identity point under their own key – the same structure registered credentials use.

Spend predicate. A depositor with registered credential $E_{\text{dep}} = (R_d, C_d)$ under their own pk_{dep} submits a SNARK proving the Chaum-Pedersen relation $C_i - sk_{\text{issuer}} R_i = C_d - sk_{\text{dep}} R_d$. That is, the two credentials decrypt to the same Identity point.

This is Theorem 6 of the proofs document (Proofs) lifted verbatim into the SNARK; a SNARK-verified instance of the CP relation implies $m_{\text{issuer}} = m_{\text{dep}}$, i.e., *same underlying Identity*.

Consequence. Only accounts whose underlying Identity scalar matches the issuer's can spend a Type 1 note. This is the Wormhole semantics: Alice mints a Type 1 note from address A_1 and later deposits at address A_2 , where both addresses carry Alice's Identity (different registered credentials, same M). Observers cannot link the two addresses beyond the Identity-constant relation they already have.

3.2 Type 2: Public Issuer Identity ("Signed Cheque")

Identity payload.

$$\text{id}_{\text{payload}}^{(2)} = (m_{\text{issuer}}, \sigma_{\text{issuer}})$$

where $m_{\text{issuer}} \in \mathbb{Z}_q$ is the issuer's Identity scalar *in plaintext* and σ_{issuer} is a Schnorr signature on cm under the issuer's registered $pk_{\text{issuer}} = m_{\text{issuer}} \cdot G$ (or a signed commitment to the note; details equivalent).

Anyone reading the commitment openings learns who issued the note: the Identity point $M_{\text{issuer}} = m_{\text{issuer}} \cdot G$ is in the clear, and since this is a public-identity account, its real-world identity is world-readable from the IdentityRegistry.

Spend predicate. The SNARK proves:

1. Knowledge of ρ such that $nf = \text{PRF}_\rho(\cdot)$
2. σ_{issuer} verifies against the registered pk_{issuer} in IdentityRegistry (SNARK-inside signature verification)
3. The depositor is *some* registered identity: Schnorr proof of knowledge of sk_{dep} for a pk_{dep} that IdentityRegistry attests to (anonymous-valid) or whose public identity is published (public)

No CP-equality is required – the issuer's identity is already known, the depositor's identity is anyone properly registered.

Consequence. Any registered identity can deposit. The issuer retains the right to spend their own note (by virtue of knowing ρ), so holders of Type 2 notes bear *printer-race risk*: if the issuer kept ρ , they can race the holder to deposit. Type 2 is appropriate for transactions that close within hours or days, not long-term holding. See section "Long-Term Holding Safety".

3.3 Type 3: Stealth-Targeted to Recipient ("Addressed Cheque")

Identity payload.

$$\text{id}_{\text{payload}}^{(3)} = (R_e, pk^*)$$

where $R_e = r_e G$ is an ephemeral published with the note and

$$pk^* = H_{\text{st}}(r_e \cdot pk_{\text{recipient}}) \cdot pk_{\text{recipient}}.$$

$pk_{\text{recipient}}$ is *not* in the payload – it would reveal the target to observers. Only the recipient, scanning notes with their own $sk_{\text{recipient}}$, can recognise a note as addressed to them: for each candidate note they compute $s = sk_{\text{recipient}} \cdot R_e$ and check whether $H_{\text{st}}(s) \cdot pk_{\text{recipient}}$ equals the note's pk^* (see Corollary 3 of Theorem 7 in Proofs).

Spend predicate. The SNARK proves knowledge of a scalar α with $\alpha \cdot G = pk^*$. By Theorem 7 this implies the prover knows the stealth secret $sk^* = H_{\text{st}}(s) \cdot sk_{\text{recipient}}$, which reduces to knowing $sk_{\text{recipient}}$ (the issuer cannot compute sk^* from $r_e, pk_{\text{recipient}}$ alone without breaking DLog).

The depositor also produces a Schnorr PoK of knowledge of sk_{dep} for their own registered pk_{dep} , binding the deposit to a compliant mint-out identity. Optionally, the recipient's credential $E_{\text{recipient}}$ can be used directly: the SNARK proves CP-equality between the decrypted $M_{\text{recipient}}$ and the depositor's registered E_{dep} , making the deposit identity-binding. The minimal-privacy variant omits this and lets any registered depositor submit the stealth-spend (suitable when the recipient wants pseudonymous redemption to a fresh account).

Consequence. Only the recipient – the holder of $sk_{\text{recipient}}$ – can spend the note. The issuer *cannot* spend what she printed (Theorem 7, Corollary 1 – "printer non-fungibility"). Third parties who see the published note learn nothing about $pk_{\text{recipient}}$. Type 3 is safe for long-term holding because the issuer's knowledge of r_e does not translate into spending power.

3.4 Comparison of the Three Bindings

Property	Type 1 Encrypted Issuer	Type 2 Public Issuer	Type 3 Stealth-Targeted
Issuer known to observers	No (ElGamal-encrypted)	Yes (plaintext)	No (not in payload)
Recipient known to observers	No (anyone with same M can spend)	No (any registered identity can spend)	No (stealth addr)
Observer links burn-mint	Only via timing; not by identity	Public flow with anonymous recipient	Not at all
Issuer can spend own note	Yes (Wormhole self-case)	Yes (knows ρ)	No (Theorem 7 Corollary 1)
Long-term holding safe	Yes (issuer = intended spender)	No (issuer race)	Yes (printer non-fungibility)
Replaces	Wormhole v1 certificates	Bank cheques, attribution payments	Cash Mode B targeted notes
Spend-circuit soundness	Theorem 6 (CP-equality)	Schnorr + signature-in-SNARK	Theorem 7 (stealth Schnorr)

The three together span every existing use case in the wormhole/cash series. The *Cash Mode A* (pure bearer, no identity hook at all) is intentionally absent from this scheme; see "What Gets Dropped" below.

4 The Interaction Matrix: Note Binding x Account Mode

The current Identity system (Identity) distinguishes two registered-account visibility modes:

- **Anonymous-valid.** The account is provably KYC-verified but the underlying Identity is encrypted. Counterparties need a bilateral **approve** handshake to learn who they are dealing with.
- **Public.** The account publishes its Identity claim in the clear. Counterparties see the real-world Identity without an **approve** step.

Permissionless accounts are excluded: they have no registered Identity and therefore cannot be either the minting side of a note nor the depositing side. Every cell below requires at least one endpoint to be a registered identity.

Crossing the three note bindings with the two account modes at deposit time produces six cells. Each row corresponds to a note *type*; each column corresponds to the *depositor account* mode.

Note type	Anonymous-valid depositor	Public depositor
1. Encrypted issuer	Wormhole, private-private Alice (issuer) and Alice' (depositor) share M ; both anonymous-valid; no link. Canonical Wormhole.	Wormhole, private-to-public Alice (issuer, private) deposits to her public account. Observers see: the public account received value from some pool note. Use: moving wealth from private trading to a public identity.
2. Public issuer	Public-to-private cheque Issuer X (public) writes a cheque; any registered anonymous-valid account cashes it. Recipient identity hidden from public. Use: payroll to private employees; charitable disbursement; B2C payments.	Public-to-public transfer by cheque Both parties public. Equivalent to a plain public transfer except the note interposes a delay/indirection. Useful: pre-signed bearer cheques, printed and mailed before recipient is determined.
3. Stealth-targeted	Private-to-private addressed cheque Issuer (any) targets a specific anonymous-valid recipient. Only recipient can spend. No public link between issuer and recipient. Use: targeted remittances, inheritance, estate transfers.	Private-to-public addressed cheque Issuer targets a public identity. The public identity is still not in the payload (stealth addr); only the recipient recognizes the note on scan. Use: donations to a public charity from private accounts without a traceable transfer.

Several observations:

1. **The Wormhole's full functionality (Type 1, Cell 1 and 2)** collapses to a single note-pool spend circuit. No burn addresses, no `_mint_noincrease`, no approval machinery for pseudo-accounts.
2. **Public-issuer cheques (Type 2)** supply the long-missing "cashier's cheque" primitive: a public issuer commits to a transfer that any registered holder can realise. This is the natural form for payroll issuance, charitable disbursements, and attribution-preserving payments that current BUCK flow lacks.
3. **Stealth-targeted cheques (Type 3)** preserve the "personal cheque" semantics from BUCK Cash's Mode B while gaining the property that the depositor can be a distinct *account* from their registered identity if desired (the SNARK binds the stealth secret, not the specific deposit address).
4. **Public-to-public transfers via Type 2 (Cell 2.2)** are a degenerate case: the same information flow could happen via a direct public **transfer**. The value of Cell 2.2 is the *timing*: the cheque can be printed, mailed, or held by the issuer before the recipient decides which of their public accounts to deposit to.

5. **Account modes are independent of issuance:** the issuer’s account visibility at mint time and the depositor’s account visibility at deposit time are chosen independently. The matrix above shows the *depositor* dimension; a full 3x2x2 enumeration would duplicate most cells without adding structural insight.

DRAFT

5 Demurrage at the Pool Level

The central simplification of this design: **the Note Pool is just a regular BUCK account.** Specifically, it is the BUCK ERC-20 contract's own balance. BUCKs transferred to the contract's address join the pool; BUCKs transferred out (as note redemptions) leave the pool. The contract account accumulates BUCK-age like every other account and participates in the ordinary demurrage accounting with no special-case code. The only piece of additional state is a single scalar F , the sum of face values of live notes.

5.1 Reminder: How BUCK Demurrage Actually Works

Demurrage is *not transferred* on any per-account basis. It is an accounting entry maintained on each account's state:

- Every block, the Jubilee account's balance grows by $\kappa \cdot \text{totalSupply} \cdot \Delta t$. This is a single global operation; the sum of all other accounts' nominal balances is held constant.
- Each account X has a running BUCK-age A_X that grows at rate B_X (its nominal balance) per unit time.
- When account X transfers v BUCK out, its spendable fraction is $(B_X - \kappa A_X)/B_X$; the outgoing transfer costs the account $v \cdot B_X / (B_X - \kappa A_X)$ of its nominal balance, and the corresponding $\kappa A_X \cdot v / (B_X - \kappa A_X)$ demurrage "owed" is simply extinguished from (B_X, A_X) against the Jubilee's autonomous growth.

System-wide invariant (preserved by every transfer):

$$\sum_{\text{all accounts } X} B_X + B_{\text{Jubilee}} = \text{constant}.$$

The Jubilee balance reflects the aggregate accrual $\kappa \int_0^t \text{totalSupply}(s) ds$; individual accounts' κA_X quantities track *their share* of that total.

5.2 The Pool Account's Demurrage

Treat the BUCK contract address as an ordinary account. Let:

- B = the contract's ERC-20 balance (= sum of all BUCK transferred in via mints and donations, minus all BUCK transferred out via redemptions, modulo the standard demurrage adjustment on outgoing transfers).
- A = the contract's accumulated BUCK-age, as maintained by the ERC-20 ledger.

B and A are already on chain; the Notes design queries them, it does not compute them. The pool's *effective spendable value* at any moment is

$$E = B - \kappa \cdot A.$$

This is the amount the contract would be able to transfer out if it emptied itself in a single call – exactly the same definition that applies to any other account.

5.3 The Deposit Formula

When a note of face v is redeemed, the contract delivers

$$v_{\text{delivered}} = v \cdot \frac{E}{F} = v \cdot \frac{B - \kappa A}{F}.$$

The interpretation is exact: each live-note face-unit is backed by the pool's effective value divided equally across all outstanding face. A deposit claims its proportional share.

Equivalently, the effective per-note demurrage rate is $1 - E/F = 1 - (B - \kappa A)/F$. In the "ideal" case where $B = F$ (every BUCK in the pool corresponds to a live note's face value, no donations or losses), this reduces to $\kappa A/B$ – exactly the pool account's own demurrage rate, which is the BUCK-wide κ scaled by the pool's average BUCK-age-per-balance.

5.4 Donations and Negative Effective Demurrage

Any BUCK transferred to the contract address *without* minting a corresponding note increases B but not F . This happens naturally in several ways:

1. **Accidental transfers.** A user fat-fingers the contract address as a recipient.
2. **Philanthropic donations.** A user deliberately sends BUCK to the contract to subsidise the pool.
3. **Recovered collateral.** Mechanism-level refunds (e.g., an issuer's collateral unlock flowing into the pool) sent to the contract address.
4. **Rounding / dust.** Accumulated fractional-unit residuals from integer arithmetic in redemption divisions.

Each such transfer raises E/F . If donations are large enough that $E/F > 1$, the next redemption delivers *more* than its face value: "negative effective demurrage". This is not a bug – it is the mathematically clean outcome of the pool being over-backed relative to its note obligations, and the excess is distributed pro-rata across live notes as they are redeemed.

The property is desirable:

- *Donations land.* Philanthropists can cheaply subsidise all outstanding noteholders. There is no accidental burn sink: a donation is consumed by the next round of redemptions.
- *Accidents self-heal.* A misdirected transfer to the contract doesn't vanish; it becomes a pro-rata benefit to live noteholders (who may include the sender).
- *Over-collateralisation is cheap to express.* An issuer wishing to guarantee "notes redeem at $\geq 1.01 \times \text{face}$ " can simply fund the pool ahead of the corresponding mints.

5.5 Conservation is Automatic

No additional Jubilee stream is emitted from the pool. The pool account participates in the BUCK system's ordinary demurrage accounting:

- The Jubilee account grows at rate $\kappa \cdot \text{totalSupply}$ per block. The pool's balance B is part of totalSupply , so the pool's share of Jubilee growth is exactly $\kappa \cdot B$ – the same rate a regular holder with balance B would contribute.

- When the contract transfers $v_{\text{delivered}}$ out, the standard ERC-20 demurrage accounting reduces (B, A) as for any other outgoing transfer.
- When users transfer BUCK to the contract, the sender pays the standard demurrage on *their* outgoing transfer; the contract receives a clean v that begins accruing age immediately.

Conservation of the system total $\sum_X B_X + B_{\text{Jubilee}} = \text{constant}$ is preserved transaction-by-transaction, with no pool-specific invariant to maintain.

5.6 Numeric Example

Suppose the pool holds $B = 10\,000\,000$ BUCK, has accumulated $A = 5\,000\,000$ BUCK-years (average BUCK-age of 0.5 yr), and $F = 10\,000\,000$ BUCK of live notes outstanding. Then $E = B - \kappa A = 10,000,000 - 0.02 \cdot 5,000,000 = 9,900,000$, and $E/F = 0.99$: every note redeems at 99% of face, independent of how long the note itself was held.

Now suppose an anonymous donor sends 500,000 BUCK to the contract address. B rises to 10,500,000; the donor's BUCK-age transfers along with the principal per standard ERC-20 semantics (add to A), but the donor's demurrage on the *outgoing* transfer is paid out of their own account, not the pool's. The new ratio is roughly $E'/F \approx 1.04$: the next redemption delivers $\sim 104\%$ of face. As redemptions drain F faster than B , the premium compresses back toward 1.0 and eventually below.

No Jubilee payment was emitted by the pool during any of this; no aggregate-level invariant was adjusted. The contract's (B, A) move through ordinary ERC-20 transitions; F moves through increments on mint and decrements on deposit.

5.7 Who Bears the Demurrage, Really

The simpler model has a sharper answer: **the current noteholder bears the demurrage that the pool accrued while holding their BUCK-equivalent.** When you mint a note of face v , you transfer v BUCK to the pool. While the pool holds that v , it accrues BUCK-age at rate v . When *any* note is redeemed, it pays its share of the pool's total BUCK-age (κA) weighted by its fraction of face (v/F).

This is not exactly "you pay for your own note's age": if the pool has high turnover, A stays small relative to B , and your per-note demurrage is correspondingly small even if you personally held for a long time. Conversely, in a stagnant pool, even a short-held note pays a larger share. The pool is a *mutualised* demurrage system among noteholders – and in the limit where you are the only noteholder, it reduces to "you pay for your own age" exactly.

6 Long-Term Holding Safety: The Printer-Trust Distinction

Pool-rate demurrage makes every note type equally cheap to *hold*. What distinguishes them for long-term storage is not demurrage but the *printer-trust* model: who, besides the intended holder, retains information that could let them spend the note?

6.1 The Printer-Trust Model

At mint time, the issuer knows:

- The note's commitment preimage $(v, \rho, \text{id}_{\text{payload}}, \text{predicate})$
- Whatever additional secret is packed into $\text{id}_{\text{payload}}$ (the issuer's sk_{issuer} for Type 1, the ephemeral r_e for Type 3, etc.)

The question is whether this knowledge enables the issuer – or anyone else with the same knowledge – to spend the note later.

6.2 Type 1 (Encrypted Issuer Identity): Safe

The spend predicate requires a Chaum-Pedersen proof of equality with the depositor's *registered* credential. The issuer is the intended eventual depositor (possibly via a different registered account sharing the same M). No third party can spend: even with full knowledge of ρ , a non-issuer's registered credential does not encrypt M_{issuer} , so the CP relation fails.

Long-term holding is safe. The only person who can spend is the issuer, and the issuer is the intended holder.

6.3 Type 2 (Public Issuer Identity): Unsafe for Long-Term Storage

The spend predicate requires only knowledge of ρ plus any registered identity. The issuer minted the note, so the issuer knows ρ . If the issuer retains any copy of ρ after printing the physical note – in a database row, a printed backup, a backup transaction – the issuer can race the holder to deposit.

This is not a cryptographic failure; it is a policy failure. A trustworthy issuer *could* destroy ρ after printing. But the *holder has no way to verify that destruction*. The note is only as safe as the issuer's operational discipline.

Long-term holding is unsafe. Type 2 notes should be cashed promptly (hours to days). They are a cheque, not a savings instrument. In the failure mode where the issuer does retain ρ , legal recourse is via the issuer's registered Identity – which is the point of making the issuer *public*: they cannot deny having printed the note, and the registered Identity is subpoena-able.

For use cases that need long-term safety with public attribution (e.g., an inheritance instrument where the issuer's identity must be known), the natural solution is to use Type 3 (stealth-targeted) and separately publish the issuer's identity in a signed accompanying statement rather than bake it into the commitment.

6.4 Type 3 (Stealth-Targeted to Recipient): Safe

The spend predicate is a Schnorr PoK of the stealth secret $sk^* = H_{\text{st}}(s) \cdot sk_{\text{recipient}}$. Computing sk^* requires $sk_{\text{recipient}}$, which only the recipient possesses. The issuer knows r_e and can compute $s = r_e \cdot pk_{\text{recipient}}$ and $H_{\text{st}}(s)$, but *cannot* compute $sk^* = H_{\text{st}}(s) \cdot sk_{\text{recipient}}$ without $sk_{\text{recipient}}$ (Theorem 7, Corollary 1 – "printer non-fungibility").

Long-term holding is safe. The issuer’s knowledge of every mint-time secret does not enable them to spend. Only the intended recipient, through their own sk , can redeem.

6.5 Summary: Which Notes Can Be Saved?

Note type	Issuer can spend later?	Long-term holding safe?	Comparable instrument
Type 1	Only to themselves	Yes	Savings account (self-owned)
Type 2	Yes (knows ρ)	No	Uncashed cheque (needs to clear)
Type 3	No (Theorem 7)	Yes	Registered bond (recipient-locked)

The consequence: *only Type 1 and Type 3 notes are suitable for long-term holding.* A user printing a Type 2 cheque and placing it in a safe deposit box for a decade is, cryptographically, trusting the issuer’s operational discipline for a decade; a user doing the same with Type 1 or Type 3 is trusting only the cryptography.

This constrains how the note types map to use cases:

- **Long-term savings, cold storage, estate, self-wormhole:** Type 1 (if you are your own beneficiary) or Type 3 (if a named heir is the beneficiary).
- **Cheques, payroll, invoices:** Type 2, cashed promptly.
- **Targeted gifts, remittances, inheritance:** Type 3.
- **Wormhole-style cross-address transfers:** Type 1 exclusively.

7 Required Identity Cryptography

The unified design re-uses the existing Identity machinery (Identity) wherever possible, adding one new note-side primitive and one re-use of the Theorem 7 stealth-address construction.

7.1 Components Already Present

The following are unchanged from Identity v1 and are used directly:

1. **Pointcheval-Sanders issuer signature** σ on the Identity scalar m . Re-randomisation continues to be used for unlinkable credential generation (Identity Fountain). No change.
2. **ElGamal registered credential** $E = (R, C) = (rG, M + r \cdot pk)$ where $M = m \cdot G$. Continues to be the on-chain identity witness. No change.
3. **Chaum-Pedersen equality of discrete logs**. Used by Type 1 spend predicate for the same- M proof; identical to the **approve** flow's CP proof, just wrapped in a SNARK. Theorem 6 applies unchanged.
4. **Schnorr PoK of sk for registered pk** . Used by Type 2 and Type 3 spend predicates to bind the depositing account to a registered identity. Single-statement case of CP, Theorems 3-5 apply verbatim.
5. **IdentityRegistry** mapping between on-chain addresses and registered credentials (PS signatures, ElGamal ciphertexts, public-claim flags).

7.2 New Primitive: The Note Commitment Hash

A SNARK-friendly commitment hash H_{cm} (Poseidon) is needed to compute the commitment inside the spend circuit. The hash is deterministic, collision-resistant over the note's field, and efficient in R1CS; Poseidon over the BN254 scalar field is the natural choice and is already used by the Aztec and Zcash Sapling circuits.

No novel cryptographic construction; only a circuit-level primitive that is standard in modern SNARK systems.

7.3 New Primitive: The Nullifier PRF

The nullifier PRF $\text{PRF}_{\text{key}}(\rho)$ is also Poseidon-based, keyed differently per type:

- **Type 1:** key = sk_{issuer} (which, per Theorem 6, equals the depositor's sk since same M). Nullifier is therefore computable by the depositor and uniquely determined by ρ and the Identity scalar shared between issuer and depositor.
- **Type 2:** key = ρ itself; nullifier = $H_{\text{nf}}(\rho)$, publicly computable by anyone who knows ρ . This is the reason Type 2 is vulnerable to issuer-race: whoever knows ρ can compute the nullifier and race a deposit.
- **Type 3:** key = sk^* (the stealth secret, known only to the recipient). The nullifier is uniquely bound to (sk^*, ρ) ; only the intended recipient can compute it.

Cross-type nullifier-space separation is enforced by a type-domain-separator byte in the PRF input, so collisions across types are avoided.

7.4 Re-Used Primitive: Theorem 7 (Stealth Address)

The stealth-address construction $pk^* = H_{st}(s) \cdot pk_{\text{recipient}}$ is proved sound in the proofs document (Theorem 7) under DLog on G_1 alone. The same construction is used here verbatim for Type 3 notes. No modification, no new assumption.

7.5 The Re-Encryption Operation at Mint

Type 1 and Type 3 require the issuer to perform a re-encryption operation at mint time:

Type 1 mint-time re-encryption. The issuer takes their own registered $M_{\text{issuer}} = m_{\text{issuer}} \cdot G$ and produces $E_{\text{issuer}}^{(\text{note})} = (r_i G, M_{\text{issuer}} + r_i \cdot pk_{\text{issuer}})$ with fresh r_i . This is just a fresh ElGamal encryption under the issuer's existing pk; the same operation Alice performs at **approve** time.

Type 3 mint-time stealth derivation. The issuer picks $r_e \leftarrow \mathbb{Z}_q^*$, computes $R_e = r_e G$, computes the shared point $s = r_e \cdot pk_{\text{recipient}}$ (the issuer can do this because she knows r_e and $pk_{\text{recipient}}$), and computes $pk^* = H_{st}(s) \cdot pk_{\text{recipient}}$. She packs (R_e, pk^*) into the commitment. She does *not* compute $sk^* = H_{st}(s) \cdot sk_{\text{recipient}}$: she can't, because she lacks $sk_{\text{recipient}}$.

Both operations are single-curve-multiplication and single-hash operations on BN254 G_1 . No pairing, no new hardness assumption.

7.6 The Re-Encryption Operation at Spend (Type 1 Only)

A Type 1 depositor must produce a fresh ElGamal encryption $E_{\text{dep}} = (r_d G, C_d) = (r_d G, M_{\text{issuer}} + r_d \cdot pk_{\text{dep}})$ and a SNARK-internal CP proof that $E_{\text{issuer}}^{(\text{note})}$ and E_{dep} decrypt to the same M . This is Theorem 6 (end-to-end re-encryption correctness) lifted into the SNARK circuit: the statement proved is identical to the on-chain CP proof from **approve**, just encoded as a SNARK witness rather than a Fiat-Shamir NIZK.

The extractable witness is $(sk_{\text{issuer}}, r_d)$, and since sk_{issuer} is bound (by registration) to M_{issuer} , the proof establishes that the depositor owns the registered credential that encrypts the same M the issuer encrypted.

7.7 The Schnorr PoK Operation at Spend (Type 2 and Type 3)

Both Type 2 and Type 3 spenders include a Schnorr PoK of sk_{dep} for their registered pk_{dep} , Fiat-Shamir bound to the spend transaction. For Type 3, an *additional* Schnorr PoK of sk^* for pk^* is required, as per Theorem 7.

Both Schnorr proofs are standard; no novelty.

7.8 Summary: New Cryptographic Components

Component	Status	Assumption
Poseidon commitment hash	New circuit primitive (standard in SNARKs)	ROM-like
Poseidon nullifier PRF	New circuit primitive	ROM-like
PS signature (issuer)	Re-used from Identity	q-SDH
ElGamal credential + re-encryption	Re-used from Identity	DLog
Chaum-Pedersen equality proof	Re-used (lifted into SNARK) from Identity	DLog + ROM
Stealth address derivation	Re-used from Proofs Theorem 7	DLog + ROM
Schnorr PoK of sk	Re-used from Identity	DLog + ROM
SNARK proof system (Groth16/PLONK)	Standard on BN254	system-dep

Only the Poseidon hash family is new, and it is already standard in the SNARK ecosystem. No novel hardness assumption is introduced.

8 The SNARK Circuits

Three spend circuits (one per note type) and one mint circuit are required. Alternatively, a single circuit conditioned on a public **type** input subsumes all three spends; this is the more efficient deployment choice but costs a larger universal verifier.

8.1 Mint Circuit

Public inputs. **totalFace** (the total face value $\sum v_i$ of the notes being minted), $\{cm_1, \dots, cm_N\}$ (commitments to be appended), and **issuerRegistryEntry** (a Merkle root or index into IdentityRegistry attesting to the issuer’s registered credential).

Witnesses. For each note i : $(type_i, v_i, \rho_i, id_{payload,i}, predicate_i)$, plus the issuer’s private material for whatever $id_{payload}$ the note requires.

Constraints.

1. $\sum v_i = \text{totalFace}$ (amount conservation).
2. Each $cm_i = H_{cm}(type_i, v_i, \rho_i, id_{payload,i}, predicate_i)$ (opening binding).
3. For each Type 1 note: $id_{payload,i}^{(1)} = (R_{i,i}, M_{\text{issuer}} + r_{i,i} \cdot pk_{\text{issuer}})$ with $R_{i,i} = r_{i,i}G$, and M_{issuer} matches the issuer’s registered ElGamal decryption.
4. For each Type 2 note: $id_{payload,i}^{(2)} = (m_{\text{issuer}}, \sigma_i)$ with σ_i a valid Schnorr signature on cm_i under $pk_{\text{issuer}} = m_{\text{issuer}}G$.
5. For each Type 3 note: $id_{payload,i}^{(3)} = (R_{e,i}, pk_i^*)$ with $R_{e,i} = r_{e,i}G$ and $pk_i^* = H_{st}(r_{e,i} \cdot pk_{\text{recipient},i}) \cdot pk_{\text{recipient},i}$, for each note’s target recipient.

Contract side. The contract verifies the proof, pulls **totalFace** BUCK from the caller (ordinary ERC-20 transfer, caller pays their own outgoing-transfer demurrage), appends each cm_i to **noteTree**, and increments **noteFaceSum** by **totalFace**. No other state is touched – the contract’s (B, A) update through the normal transfer path.

8.2 Spend Circuit (Type 1 – Encrypted Issuer)

Public inputs. **noteRoot** (current Merkle root of **noteTree**), **nullifier**, **faceValue** (the note’s face v , revealed), **depositorRegistryEntry** (pointer into IdentityRegistry for the depositor’s credential).

Witnesses. $(v, \rho, id_{payload,i}^{(1)}, path)$, $sk_{\text{issuer/dep}}$ (shared, by Theorem 6), the depositor’s fresh encryption randomness r_d , and the depositor’s registered credential $E_{\text{dep}}^{(\text{registry})}$.

Constraints.

1. $cm = H_{cm}(1, v, \rho, id_{payload,i}^{(1)}, pred)$ opens to a leaf of **noteTree** under **path** (Merkle inclusion).
2. $nullifier = H_{nf}(1 || sk_{\text{dep}} || \rho)$.
3. $id_{payload,i}^{(1)} = (R_i, C_i)$; the registered credential $E_{\text{dep}}^{(\text{registry})} = (R_d^{\text{reg}}, C_d^{\text{reg}})$ exists in IdentityRegistry; CP-equality $C_i - sk_{\text{dep}}R_i = C_d^{\text{reg}} - sk_{\text{dep}}R_d^{\text{reg}}$ holds (Theorem 6).
4. **faceValue** = v (witness v revealed as public input).

The contract checks the nullifier is unused, reads its own (B, A) and $F = \text{noteFaceSum}$, computes $\text{delivered} = v \cdot (B - \kappa A)/F$, transfers delivered BUCK to the recipient via standard ERC-20, and decrements noteFaceSum by v . The demurrage arithmetic lives entirely in the contract, not the SNARK.

8.3 Spend Circuit (Type 2 – Public Issuer)

Public inputs. noteRoot , nullifier , faceValue , issuerPkClaim (the issuer’s public-identity claim for human-readable verification, redundant with the circuit’s own check but useful for wallets), $\text{depositorRegistryEntry}$.

Witnesses. $(v, \rho, \text{id}_{\text{payload}}^{(2)}, \text{path})$, sk_{dep} , the depositor’s Schnorr nonce k_{dep} .

Constraints.

1. Merkle inclusion of cm under path (as Type 1).
2. $\text{nullifier} = H_{\text{nf}}(2 \parallel \rho)$ (publicly derivable from ρ – this is the first-come-first-served property of Type 2).
3. $\text{id}_{\text{payload}}^{(2)} = (m_{\text{issuer}}, \sigma_{\text{issuer}})$ with σ_{issuer} verifying against $pk_{\text{issuer}} = m_{\text{issuer}}G$ in IdentityRegistry.
4. Schnorr PoK of sk_{dep} for pk_{dep} in IdentityRegistry, Fiat-Shamir bound to msg.sender , chainid , nullifier .
5. $\text{faceValue} = v$.

The contract’s delivery computation is identical to Type 1: $\text{delivered} = v \cdot (B - \kappa A)/F$, computed outside the SNARK.

8.4 Spend Circuit (Type 3 – Stealth-Targeted)

Public inputs. noteRoot , nullifier , faceValue , $\text{depositorRegistryEntry}$. pk^* is *not* a public input: revealing it would let an observer look up $pk_{\text{recipient}}$ in IdentityRegistry and deanonymise the target. Instead, pk^* is a private witness, and the Merkle inclusion proof binds it to the commitment.

Witnesses. $(v, \rho, R_e, pk^*, \text{path})$, $sk_{\text{recipient}}$, the stealth secret $sk^* = H_{\text{st}}(sk_{\text{recipient}} \cdot R_e) \cdot sk_{\text{recipient}}$, sk_{dep} , Schnorr nonces.

Constraints.

1. Merkle inclusion of $cm = H_{\text{cm}}(3, v, \rho, (R_e, pk^*), \text{pred})$.
2. $\text{nullifier} = H_{\text{nf}}(3 \parallel sk^* \parallel \rho)$.
3. $sk^* \cdot G = pk^*$ (Theorem 7 completeness).
4. $sk^* = H_{\text{st}}(sk_{\text{recipient}} \cdot R_e) \cdot sk_{\text{recipient}}$ (Theorem 7 setup; also binds $pk_{\text{recipient}}$ indirectly via this relation).
5. Schnorr PoK of sk_{dep} for pk_{dep} in IdentityRegistry.
6. $\text{faceValue} = v$.

As with Types 1 and 2, the contract computes $\text{delivered} = v \cdot (B - \kappa A)/F$ outside the SNARK.

Optional identity-binding. If the recipient wishes to bind the deposit to their registered identity (for tax/audit purposes), the circuit additionally proves CP-equality between $E_{\text{dep}}^{(\text{registry})}$ and a re-encryption of the Identity point M decrypted from $E_{\text{recipient}}^{(\text{registry})}$, matching the recipient’s own credential. This variant is opt-in; the minimal-privacy variant simply requires *any* registered depositor.

8.5 Circuit Engineering Budget

Ballpark constraint counts (orders of magnitude, BN254 Groth16):

Circuit element	Approx. constraints
Poseidon commitment hash (1 call)	~250
Poseidon nullifier PRF (1 call)	~250
Merkle path verification (20 levels)	~5000
ElGamal / Chaum-Pedersen (Type 1)	~2000
Schnorr PoK (Types 2/3)	~500
Stealth-address relation (Type 3)	~1500
Range check on v	~100

Total spend-circuit constraint count is in the range 6,000-10,000, comparable to Zcash Sapling’s Spend circuit (~25,000 including note-commitment randomness features we don’t need) and Aztec’s basic note-spend circuit. On-chip proving time on a modern smartphone: a few seconds.

8.6 Common Circuit Gadgets

A base-level library of gadgets is shared across all spend types:

- Merkle inclusion (Poseidon path)
- Poseidon hash wrapper
- Schnorr PoK
- Range proofs on field elements (for v)

Type-specific gadgets layer on top. This is standard SNARK library architecture (Aztec Noir, Circom, Halo2 all support this layering).

9 Migration from v1 Wormhole and BUCK Cash

The transition is orderly and preserves all existing cryptographic anchors.

9.1 Wormhole v1 to Notes

A v1 burn-address certificate $(addr, v, age)$ maps to a single Type 1 note:

```
migrate_v1_to_notes(addr, v, age):
    sk_addr      = recovered Schnorr key bound to addr
    pk_addr      = sk_addr * G
    M_addr       = registered M for addr (from IdentityRegistry)
    r_i          = fresh random
    R_i, C_i     = ElGamal(M_addr, r_i, pk_addr)
    rho          = fresh random
    cm           = Poseidon(1, v_effective, rho, (R_i, C_i), None)
    # v_effective = v - kappa * age folds the certificate's
    # accrued BUCK-age into its face value at migration time.
    submit mint proof: burn (addr, v, age), append cm to noteTree
```

The v1 certificate's accumulated demurrage is discharged at migration (by computing the certificate's current effective value $v_{\text{eff}} = v - \kappa \cdot \text{age}$ and using that as the note's face). After migration the note participates in the pool as a normal Type 1 note; its subsequent demurrage is the pool-rate, not the per-certificate rate. The v1 burn address is emptied; its BUCK (net of demurrage) flows into the contract balance B exactly as any other mint.

9.2 BUCK Cash to Notes

- **Cash Mode B (stealth-targeted)** maps one-to-one to Type 3; the cryptographic structure is identical.
- **Cash Mode C (programmable)** can be expressed as a Type 1, 2, or 3 note with a non-trivial `predicate` field in the commitment, evaluated by the spend circuit. The three base types subsume the most common programmable variants (time locks are a predicate on t_{mint} ; oracle conditionals are a predicate on oracle state; multi-sig is a conjunction of multiple Schnorr PoKs).
- **Cash Mode A (pure bearer)** does not map to any of the three types. See "What Gets Dropped" below.

9.3 v1 Coexistence During Transition

The unified Notes architecture can be deployed alongside the v1 Wormhole during a transition window. The v1 burn-address pool remains a valid anonymity set; new users mint into the Notes pool; v1 holders migrate at their convenience. The Notes contract's `noteFaceSum` reflects only Notes-pool activity; v1 certificates continue to accrue per-certificate demurrage as before, via their individual burn-address accounts.

The final *old* v1 certificate need not be migrated – it can continue to be redeemed against its burn-address anchor indefinitely, at the cost of the contract maintaining two verifier paths. A practical deadline (e.g., 5 years from Notes launch) is reasonable.

10 What Gets Dropped: Pure Bearer

The Cash document's *Mode A – bearer note with QR code sealed under a hologram, anyone who possesses the QR can deposit* does not map cleanly onto the unified scheme. The reason is the same as why Mode A in the Cash design required a physical hologram seal: a purely secret-based predicate (`know(rho)` with no identity hook) has no cryptographic mechanism preventing a malicious issuer from retaining ρ and racing the holder.

10.1 Why Mode A Is Absent

Three routes considered:

1. *Blind minting* – the issuer does not know ρ at mint time. Requires the holder to participate in minting (giving a blinded commitment). Incompatible with the "print a stack of bearer notes and hand them out" use case.
2. *Trusted printer* – hardware security module or audited print service that provably erases ρ . Works in principle; undermines the "trustless bearer instrument" pitch.
3. *Short expiry* – Mode A notes expire after a bounded period, limiting the issuer-race window. Equivalent to Type 2 with a short-term contract; the use case is then cheque-like, not cash-like.

None of these route options preserve the "\$20 bill-equivalent" property that Mode A was designed for. The unified scheme therefore *does not attempt* to provide a pure-bearer cryptographic note, and defers that use case to a separate design dimension:

10.2 The Physical-Token Escape Hatch

A pure-bearer instrument remains available via the *physical* trust model that all physical bearer instruments already rely on. Specifically: an issuer operating a print service with a hardware-attested erasure of ρ can issue Type 2 notes with a public warranty that the service did in fact erase ρ . The warranty is an off-chain social/legal claim, not a cryptographic one, but it is identical in kind to the trust extended to the Bank of Canada when it issues physical polymer banknotes (whose security rests on BoC printing discipline, not on cryptography).

This keeps the pure-bearer door ajar without requiring the unified scheme to pretend it solves the cryptographic problem of making a truly trustless bearer note. BUCK Cash's original Mode A was arguably already conceding this point via the tamper-evident seal ("the seal makes it safe to display, but any unsealed QR is spendable by anyone"): the seal is a physical-world primitive, not a cryptographic one.

10.3 Use-Case Coverage Without Mode A

The canonical Mode A use cases re-route cleanly:

Use case	Cash Mode A	Unified scheme
Tip jar	Printed Mode A sheet	Public-issuer Type 2 cheques, held briefly
Walking-around money	Printed Mode A notes	Self-issued Type 1 in a hot wallet
Gifts to unknown recipient	Printed Mode A	Type 2 signed cheque, short expiry
Disaster preparedness cold	Printed Mode A hoard	Self-issued Type 1, long-term safe
Underbanked payment rail	Physical Mode A	Type 2 cheque from issuer's office

Every use case that truly requires "no identity link at all" is handled by the physical-token escape hatch above. Every use case that tolerates an *encrypted* identity link (which is most of them) gains stronger cryptographic guarantees via Types 1 and 3.

11 Security Assumptions and Theorems Used

The unified Notes design rests on the same cryptographic foundations as the existing Identity system, plus one already-proved additional theorem:

Primitive / Theorem	Usage in Notes	Assumption
PS signature (issuer credential)	Mint-circuit issuer-binding	q-SDH
ElGamal on G_1	Type 1 identity payload	DLog on G_1
Theorem 6 (CP re-encryption)	Type 1 spend predicate	DLog + ROM
Theorem 7 (stealth-targeted)	Type 3 spend predicate	DLog + ROM
Schnorr PoK (sigma protocols)	Type 2, Type 3 depositor check	DLog + ROM
Poseidon hash/PRF (SNARK-side)	Commitments, nullifiers	ROM-like (circuit)
SNARK proof system (e.g., PLONK)	Wrapping all of the above	system-specific
Merkle tree (noteTree)	Commitment anchor	collision-resistance

No *novel* cryptographic assumption is introduced. The only new primitive not already in BUCK is the Poseidon hash family, which is standard in SNARK deployments (Aztec, Zcash Sapling/Orchard, Starknet, Mina) and well-studied.

11.1 Demurrage Conservation as a Contract Invariant

Because the pool is a regular BUCK account, no Notes-specific demurrage invariant is needed: the system-wide BUCK invariant

$$\sum_{\text{all accounts}} B_X + B_{\text{Jubilee}} = \text{constant}$$

covers it automatically. The Notes contract only needs to maintain two auxiliary local invariants:

1. $F \geq 0$ (no negative note face sum).
2. Every v appearing in a deposit equals the v that appeared in the corresponding mint (enforced by the SNARK via the Merkle inclusion of cm binding v).

Both are trivial to prove from the contract's state machine; no queueing-theoretic assumption is required. The arithmetic $\text{delivered} = v \cdot (B - \kappa A) / F$ is a simple contract-side computation against on-chain ERC-20 state.

12 Implementation Scope

Rough engineering estimate, assuming the existing Alberta Buck contract skeleton and an existing SNARK toolchain (Aztec Noir or Circom/snarkJS):

12.1 Smart Contract Additions

- **noteTree** (Merkle tree with Poseidon nodes): ~1 week, bulk of code is existing Aztec/Zcash sapling implementations.
- **nullifiers** mapping: trivial.
- **noteFaceSum** scalar + mint/deposit updates: trivial.
- Deposit delivery formula $v(B - \kappa A)/F$: trivial, uses existing ERC-20 (B, A) getters.
- Three spend-circuit verifiers (Groth16-style on-chain): ~1 week integration per circuit, bulk is circuit-side.
- Mint-circuit verifier: ~1 week.
- Migration helper for v1 certificates: ~1 week.

Total: ~8 weeks for an experienced Solidity engineer.

12.2 SNARK Circuits

- Mint circuit: ~2-3 weeks (Poseidon commitments, ElGamal encryption, Schnorr signature verification inside the circuit).
- Spend circuits (three, with shared gadgets): ~4-6 weeks.
- Common gadgets library (Merkle, Poseidon wrappers, Schnorr, range checks): ~2 weeks.

Total: ~10 weeks for an experienced SNARK engineer.

12.3 Wallet & UX

- Note scanning / recognition for Type 3 (stealth-address scan): ~1 week.
- Type 1 self-mint-to-private-address flow (Wormhole replacement): ~1 week.
- Type 2 cheque creation and deposit UX: ~2 weeks.
- Type 3 targeted-note UX: ~2 weeks.
- Prover service integration for thin clients: ~2-3 weeks.

Total: ~8 weeks for a wallet team.

12.4 Cross-Checks and Audit

- Cryptographic review of the unified spend circuits: 4 weeks, external auditor with SNARK-circuit expertise.
- Economic model validation (pool-rate demurrage in stress scenarios: low turnover, high turnover, mint/deposit mismatch): 2 weeks.
- End-to-end fuzz and property-based testing: 2 weeks.

Total: ~8 weeks of external + internal audit.

Overall program estimate: 6-9 months of focused work to ship a production-quality unified Notes system, subsuming Wormhole v1 and the BUCK Cash proposal into a single architecture.

DRAFT

13 Open Questions

The unified design resolves most of the architectural redundancy but leaves a handful of economic and cryptographic questions worth sharpening before commitment:

13.1 Donation-Driven Premia and Their Incidence

Because E/F can exceed 1 whenever donations accumulate faster than redemptions drain them, the redemption rate seen by users is subject to external "weather". A large donation immediately before a redemption benefits that single depositor; a large donation spread across many redemptions benefits all of them proportionally. This is equitable *in expectation* but bursty in any given window.

Whether to smooth the bursts – e.g., by rate-limiting the fraction of $E/F > 1$ a single redemption can claim, or by time-averaging B – is a design question. The simplest answer is to do nothing: the pool is transparent, the premium math is public, and the on-chain sequencing is first-come.

13.2 Stranded Value on Lost Notes

A lost note contributes to F indefinitely. Its share of B is never claimed by the intended redeemer; instead, the pool's BUCK-age continues to accrue against that unclaimed face value, and future redeemers pay demurrage partially on behalf of the lost note. As A/F creeps upward with unreclaimed value, E/F drifts downward; holders of live notes see a slowly growing demurrage tax until the lost face value washes out.

A reclaim path is possible: after a very long period (say 99 years), the contract could treat F as including only notes whose commitments are demonstrably recent, via a rolling window. Lost value then re-surfaces as donation-equivalent at the window boundary. This requires careful circuit and economic-policy design and is deferred.

13.3 Integer-Arithmetic Rounding

The deposit formula $v \cdot (B - \kappa A)/F$ involves integer division. Residuals left behind by truncated division remain in B (they are never transferred out), raising E/F for the *next* depositor – the rounding is self-healing rather than drift-accumulating. Choice of rounding direction (floor vs. banker's) is a minor contract-level decision with no cryptographic impact.

13.4 Predicate Programmability vs. Circuit Simplicity

Keeping three fixed types simplifies the circuit count. Adding programmable predicates (time locks, oracle conditions, multi-sigs) re-introduces the complexity of Cash Mode C. A compromise: support a *bounded* predicate language in the circuit (time-lock + oracle-condition + up-to-3-sig) rather than a fully general one. The bounded language covers 90%+ of the use cases Mode C was designed for while keeping the verifier compact.

13.5 Universal Spend Circuit vs. Three

Deploying three distinct spend verifiers costs gas per transaction (one verifier per call). A universal verifier that takes `type` as a public input and branches internally costs a larger but single verifier. Gas trade-off depends on production transaction mix; if Type 1 dominates (Wormhole-style), three verifiers are cheaper. Empirical question, resolvable at deployment time.

13.6 Regulatory Interaction with Redemption Premia

Jurisdictional regulators (FINTRAC, OSFI) have rules around bearer instruments and tax treatment of negative-interest accounts. BUCK’s on-chain demurrage was argued through separately, and the Notes contract inherits that treatment with no pool-specific novelty – the contract is just another BUCK account. The one new wrinkle is that donations can push $E/F > 1$, producing redemption premia that look economically like small interest payments. Whether these count as ordinary income, capital gain, or bearer-instrument appreciation is a tax-counsel question rather than a protocol one. Not blocking but worth flagging.

DRAFT

14 Summary

The Wormhole and BUCK Cash designs converge naturally on a single note-pool primitive. Three Identity-binding types – Encrypted-Issuer (Type 1, Wormhole-style), Public-Issuer (Type 2, signed cheque), Stealth-Targeted (Type 3, addressed cheque) – together span every use case in the existing series while removing the burn-address / `_mint_noincrease` machinery, the per-certificate Identity registration, and the separate Cash Mode A/B/C taxonomy.

Demurrage needs no new machinery: the Note Pool is literally the BUCK ERC-20 contract’s own balance – a regular BUCK account with its own B and A in the ERC-20 ledger. The only Note-side state beyond the Merkle tree and nullifier set is one scalar F , the total face value of live notes. Redemption of face v delivers $v \cdot (B - \kappa A) / F$: the note’s proportional share of the pool’s effective spendable value. Conservation falls out of the existing BUCK accounting – the Jubilee grows autonomously at $\kappa \cdot \text{totalSupply}$ per block, with no Notes-specific obligation stream. Donations (deliberate, philanthropic, or accidental) raise B without raising F , reducing or inverting the effective demurrage subsequent redeemers pay – a useful property, not a bug.

The printer-trust distinction produces a sharp partition of which note types are safe for long-term holding: only *Type 1* (issuer = beneficiary) and *Type 3* (stealth-targeted to beneficiary) can be held for arbitrary durations. *Type 2* is cryptographically a short-term instrument regardless of how long a user might wish to keep it, because the issuer retains the spending secret. This is an intrinsic property that cannot be remedied by additional cryptography; it follows directly from the Type 2 identity payload carrying the issuer’s public identity and no recipient-binding.

Pure-bearer notes (Cash Mode A) are not replaced by any cryptographic Type; they remain the domain of physical trust models (tamper-evident seals, trusted printers). This is a philosophical refinement of the Cash proposal rather than a retreat: Cash Mode A already relied on a physical hologram for its security, and the unified design makes the physical/crypto boundary explicit.

No novel cryptographic assumption is introduced. Theorems 6 and 7 of the Proofs document (Proofs) apply verbatim to the Type 1 and Type 3 spend predicates, respectively; Type 2 reduces to Schnorr. The only new circuit-level primitive is Poseidon, which is already standard across the SNARK ecosystem. Implementation scope is 6-9 months for a focused team.

The unification is, in the small, a refactor that removes redundancy. In the large, it crystallises the Alberta Buck design around a single cryptographic primitive (the note pool) whose Identity-binding dimension produces the full range of financial instruments the series aspires to, and whose economic dimension (pool-rate demurrage) elegantly solves the lost-note problem that motivated the original BUCK Cash demurrage discussion.